

B b1 = new B();

b1.setx(10);  
b1.sety(20);  
b1.display();

Handwritten notes in a circle:  
x, y  
setx  
sety  
display  
b1

## Constructors in Java

A **constructor** in Java is a special member method that is automatically called when an object of a class is created. It is used to **initialize the object**.

### Key Characteristics of Constructors

- Same name as class**  
The constructor name must be exactly the same as the class name.
- No return type**  
Constructors do not have any return type, not even void.
- Called automatically**  
It is invoked automatically when an object is created using the new keyword.
- Used to initialize object variables**  
Constructors help assign initial values to instance variables.

### Syntax

```
class ClassName {
    ClassName() {
        // constructor body
    }
}
```

### Types of Constructors in Java

#### 1. Default Constructor

- A constructor with **no parameters**
- Provided automatically by Java if no constructor is written

```
class Student {
}
```

#### 2. No-Argument Constructor (User-Defined)

- Created by programmer
- Does not accept parameters

```
class Student {
    Student() {
        System.out.println("Student Created");
    }
}
```

#### 3. Parameterized Constructor

- Accepts parameters
- Used to initialize object with specific values

```
class Student {
    String name;
    Student(String n) {
        name = n;
    }
}
```

### Constructor vs Method

| Feature     | Constructor       | Method                |
|-------------|-------------------|-----------------------|
| Name        | Same as class     | Any valid name        |
| Return type | No return type    | Must have return type |
| Purpose     | Initialize object | Perform operations    |
| Called      | Automatically     | Called manually       |

Student s1 = new Student();

### Example Program

```
class Student {
    String name;
    int age;
    Student(String n, int a) {
        name = n;
        age = a;
    }
    void display() {
        System.out.println(name + " " + age);
    }
    public static void main(String[] args) {
        Student s1 = new Student("Rahul", 20);
        s1.display();
    }
}
```

Output:

Rahul 20

### Important Points (Exam-Oriented)

- Constructor initializes objects
- Constructor name must match class name

- No return type
- Called automatically when object is created
- Can be overloaded
- Cannot be inherited
- Used with new keyword

## Polymorphism in Java

Polymorphism means "many forms".

In Java, polymorphism allows a single method, object, or operator to behave differently in different situations.

In simple words:

One name → Multiple behaviors

## Types of Polymorphism in Java

There are two main types:

1. Compile-Time Polymorphism (Static Polymorphism)
2. Run-Time Polymorphism (Dynamic Polymorphism)

### 1. Compile-Time Polymorphism (Method Overloading)

This occurs when multiple methods have the same name but different parameters in the same class.

Example:

```
class MathOperation {
    int add(int a, int b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
}

class Main {
    public static void main(String[] args) {
        MathOperation obj = new MathOperation();
        System.out.println(obj.add(5, 10));
        System.out.println(obj.add(5, 10, 15));
    }
}
```

Output:

15  
30

This is called **Method Overloading**.

### 2. Run-Time Polymorphism (Method Overriding)

This occurs when a **child class provides its own implementation** of a method defined in the parent class.

Achieved using **Inheritance**.

Example:

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

class Main {
    public static void main(String[] args) {
        Animal obj = new Dog();
        obj.sound();
    }
}
```

Output:

Dog barks

This is called **Method Overriding**.

### Real-Life Example

Person → can behave as:

- Teacher
- Student
- Employee

Same person → different roles → Polymorphism

### Method Overloading vs Method Overriding

| Feature                | Method Overloading       | Method Overriding      |
|------------------------|--------------------------|------------------------|
| Occurs in              | Same class               | Parent and Child class |
| Parameters             | Must be different        | Must be same           |
| Return type            | Can be same or different | Must be same           |
| Inheritance required   | No                       | Yes                    |
| Compile time / Runtime | Compile Time             | Runtime                |

### Advantages of Polymorphism

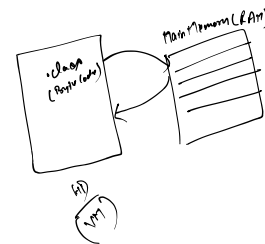
- Code reusability
- Flexibility
- Improves readability
- Supports dynamic method binding

### Important Exam Points

- Polymorphism means many forms
- Achieved using:
  - Method Overloading
  - Method Overriding
- Overloading → Compile time polymorphism
- Overriding → Runtime polymorphism
- Runtime polymorphism uses inheritance

MathOperation m = new MathOperation();

m.add(10, 10);  
m.add(10, 10, 10);  
m.add(4);  
m.add(4, 5);  
m.add(4, 5);



```
if (ch == 'A')
    obj = new Animal();
else
    obj = new Dog();

obj.sound();
```

## Abstraction in Java

Abstraction means hiding implementation details and showing only essential features to the user.

In simple words:

What to do → shown

How to do → hidden

## Real-Life Example

Example: ATM Machine

You can:

- Withdraw money
- Check balance

But you don't know:

- How the ATM processes internally

This is **abstraction**.

## How Abstraction is Achieved in Java

Abstraction is achieved using:

1. **Abstract Class**
2. **Interface**

### 1. Abstract Class

An **abstract class** is a class declared using the abstract keyword.

It can have:

- Abstract methods (without body)
- Normal methods (with body)

**Syntax:**

```
abstract class ClassName {  
    abstract void methodName();  
}
```

**Example:**

```
abstract class Animal {  
    abstract void sound();  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
    }  
}
```

## Important Points about Abstract Class

- Declared using abstract keyword
- Cannot create object directly
- Can have abstract and normal methods
- Must be inherited

### 2. Interface

An **interface** is used to achieve complete abstraction.

**Syntax:**

```
interface Shape {  
    void draw();  
}
```

**Example:**

```
interface Shape {  
    void draw();  
}  
  
class Circle implements Shape {  
    public void draw() {  
        System.out.println("Drawing Circle");  
    }  
}
```

## Abstract Class vs Interface

| Feature              | Abstract Class    | Interface                     |
|----------------------|-------------------|-------------------------------|
| Methods              | Abstract + Normal | Only abstract (traditionally) |
| Keyword              | abstract class    | interface                     |
| Inheritance          | extends           | implements                    |
| Multiple inheritance | No                | Yes                           |

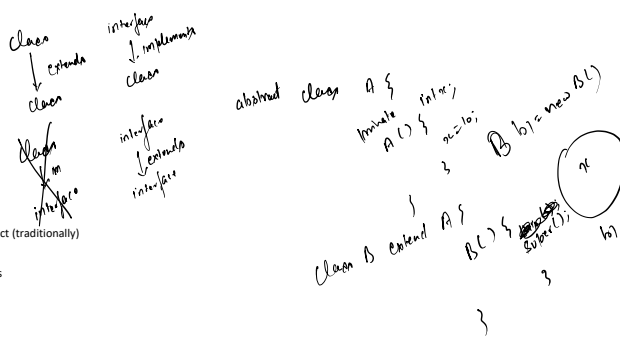
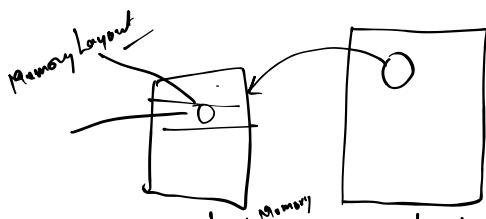
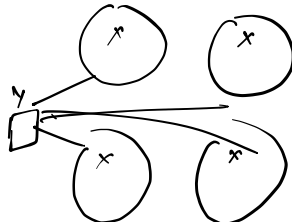
## Advantages of Abstraction

- Hides internal implementation
- Improves security
- Reduces complexity
- Improves maintainability
- Supports loose coupling

## Important Exam Definition (2 Marks)

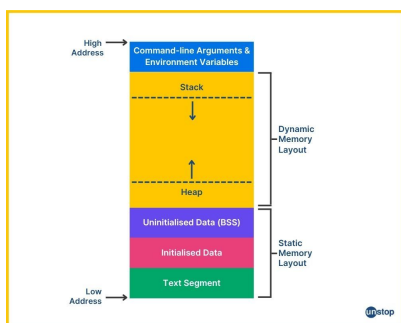
Abstraction is the process of hiding implementation details and showing only essential features to the user. It is achieved using abstract classes and interfaces in Java.

```
class A {  
    int x;  
    static int y;  
}
```



RAM / Main

HD / VM



Main.java  
`sh(Hellow world);`  
 javac fileName ByteCode

Xyz.java  

```

A {
    B {
        psvm();
        sop(main);
    }
}
  
```

A-class  
 B-class

javac xyz.java  
 java B

S =

NS: 00000101  
 11111010

$$-1 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$

$$-128 + 64 + 32 + 16 + 0 + 0 + 2 + 0$$

a = 3;

a = a \* 2 + 1 ;

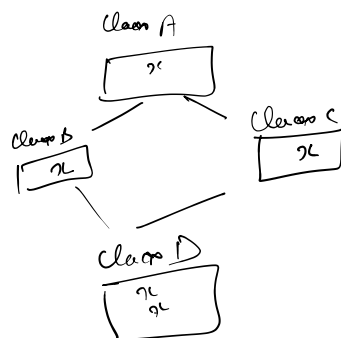
$$a = 2 + 1$$

$$a = 3$$

int x = 5;

$x \xrightarrow{5} x \xrightarrow{6} x \Rightarrow$

$x \xrightarrow{5} x \xrightarrow{7} x \Rightarrow (12)$



1. I class ABC

Error

class xyz extends ABC